Detecting Semantic Laundering in Agentic AI

A Working Paper on TAO

Jorge Perdomo
jorgeperdom@gmail.com

0.12 · 2026-05-17 · Working draft

Abstract

Agentic AI systems take actions in the world — calling tools, accessing data, communicating with people, moving resources. Today, when one of these systems misbehaves, reconstructing what happened is expensive, ad-hoc, and unreliable. The agent's own telemetry shows what its software claims it did. System logs show what its calls mechanically produced. No standard layer asks whether the agent's label for its action matches what the action actually accomplished. We call the gap between the two *semantic laundering* — a benign label sitting on top of effects that don't support it.

This paper introduces TAO, a behavioral audit substrate that closes part of the gap. TAO does not solve AI governance. It standardizes one missing evidentiary object: the comparison between what an agent says it did and what an independent observer records, under declared coverage and policy assumptions. The substrate is positioned as one layer in a multi-layer defense — necessary, not sufficient. It gives management-based regulation of agentic AI a portable behavioral evidence format without mandating any particular model architecture, without requiring access to model weights, and without slowing down deployment cycles. This paper presents the substrate's design, four worked examples, four limits exhibits, the theoretical positioning, the substrate's named falsifiers, and the open questions that remain.

---

1. The problem

A code agent running in a continuous-integration environment is asked to draft release notes for an upcoming software release. The agent has read access to a private repository and outbound network capability. It produces release notes. Three weeks later, a security team discovers that the same agent, in the same session, also sent a copy of the repository to an external endpoint nobody had authorized.

This is not a hypothetical. Patterns like it appear in every recent class of AI-deployment incident — from chatbots advising customers in ways the operator did not authorize, to coding assistants making changes to security-relevant files without notice, to support agents quietly modifying account settings. The post-incident question is the same in each case: *what did the AI actually do?* And the answer requires expensive reconstruction work from logs in whatever shape the operator's stack happened to produce.

The frustrating thing about these incidents is that the evidence existed at the time. The agent's own software emitted telemetry about what it was doing. System-level logs captured the calls that mechanically occurred. The mismatch between the two

— the agent reported “draft release notes,” the system recorded “HTTPS POST to an external endpoint” — was *visible* if anyone had been looking. What was missing was not data. What was missing was a standard layer asking the comparison question.

We call this gap **semantic laundering**: an action wearing a benign label that the action’s mechanical effects don’t actually support. The label is what the agent’s adapter emits — “this is a content-generation task” — and the effects are what observably happened — a read of a private repository followed by external network egress. The pattern shows up across domains. A customer-service agent labels a refund denial “policy enforcement” when the cited policy doesn’t actually apply. A robo-advisor labels a leveraged-ETF recommendation “suitable for retirement planning” when the client’s profile excludes leveraged products. A patient-facing triage AI labels itself as “providing general health information” while issuing dosage advice.

In each case, the agent’s representation of its own action obscures what the action mechanically did. And in each case, for the mechanically visible class of laundering, the cost of detecting it as the action happens is much lower than the cost of reconstructing it forensically months later from whatever logs the operator’s stack happened to produce.

The proposal in this paper is that closing this gap requires a substrate — not a specific technology, not a specific regulatory rule, but a standardized form for behavioral records that any regulated party can produce and any reviewer can read. We call this substrate TAO, for *behavioral audit interface for agentic AI*.

2. What TAO is

TAO is **a standard interface for comparing an agent’s declared action against independently observed effects, so operators, auditors, and regulators can detect when the label on behavior stops matching the behavior itself.**

The substrate has three pieces.

A two-layer model of action. Every action takes a structured form combining (a) a *semantic claim* about what the action was — a verb in a controlled vocabulary, plus a stated purpose and an authority — and (b) one or more *mechanical effects* drawn from a small fixed vocabulary describing what the action did to resources, capabilities, information, or commitments. The verb vocabulary is small (29 verbs in 12 families) so a reviewer can learn it. The effect vocabulary is smaller still (nine effect types in four categories) so cross-domain comparison is tractable. Every verb in the vocabulary has a published mapping naming which effect types it *requires*, which it *permits*, and which it *forbids*.

The Claim-Check Delta. A second tuple is emitted by an *independent observer* — a separate process, ideally with its own privilege boundary, that records what the action mechanically did. The Claim-Check Delta is a defined comparison between the agent's claim tuple and the observer's check tuple. It produces three results, each one of *consistent*, *inconsistent*, or *indeterminate*: a semantic-mechanical check (do the observed effects fall in the verb's mapping?), a teleological check (does the stated purpose match the observed effects?), and a factual check (does the cited authority actually authorize what happened?). When the three checks agree on *inconsistent*, the substrate has detected a structural mismatch that no single tuple alone could surface.

Mission Profiles. A Mission Profile is a deployment-specific policy document, attached to an agent, that names which actions require escalation, which are blocked, and how the agent's mappings may legitimately deviate from the published baseline. Profiles are signed, versioned, and inspectable. Override discipline — when a profile weakens a default, it must say so and explain why — makes the legitimate cases of policy deviation distinguishable from the laundering cases.

Mission Profiles are not a new institutional category. Every regulated domain already has the equivalent under different names — Rules of Engagement in military contexts, clinical protocols and scope-of-practice rules in hospitals, standard operating procedures in factories, treaties and conventions with specified escalation ladders in international relations. These rules already exist as paper documents, training materials, and case law. The substrate's contribution is to make them machine-readable so the same rules a human staff member is trained on become the rules the AI system actually follows, with the same audit-grade record produced when a decision is made.

Molecules: combining elements into governable patterns

The vocabulary is the periodic table. A specific action is a *molecule* — a particular combination of verb, target, effects, context, and justification that constitutes a specific, governable event. The same mechanical effect can be utterly different actions depending on the molecule it appears in.

Consider four molecules, all containing `RESOURCE.DAMAGE`:

- *Mass harm:* `HARM.DAMAGE.STRIKE`, target_scope `CLASS`, `RESOURCE.DAMAGE` against a protected population, consent `ABSENT`, authority `ILLEGITIMATE`, reversibility `IRREVERSIBLE`.
- *Lawful defense:* `PROTECT.DEFEND.OTHER`, target an individual attacker, `CAPABILITY.RESTRICT` + `RESOURCE.DAMAGE` with harm acknowledged, consent `IMPLICIT` (from the defended party), authority `VERIFIED`, urgency `EMERGENCY`.

- *Medical intervention:* `PROTECT.HEAL.TREAT`, target the patient, `RESOURCE.TRANSFER` (medication) + `CAPABILITY.RESTRICT` (temporary), consent `EXPLICIT`, authority `VERIFIED` (licensed physician), vulnerability `HIGH`, reversibility `REVERSIBLE`.
- *Assault disguised as care:* `PROTECT.HEAL.TREAT` (claimed), but the observed effects are `RESOURCE.DAMAGE` without `RESOURCE.TRANSFER`, consent `COERCED`, authority `CLAIMED` rather than verified.

The same elements; four different molecules; four different governable actions. The first is mass harm, the second lawful defense, the third appropriate medical care, the fourth the case the Claim-Check Delta catches when the molecule's stated label fails to match its actual structure. Rules over *molecular patterns — "block any tuple where* `HARM.*` *+ consent* `ABSENT` *+ vulnerability ≥* `HIGH`*"* — are enforceable in a way *"be safe"* is not.

The analogy is operational rather than literal. Unlike chemical molecules, TAO molecules are partly *institutional objects*: consent, authority, legitimacy, beneficiary, vulnerability, and scope are claims resolved against records, law, and domain policy, not physical properties. The substrate makes those resolutions explicit and inspectable; it does not make them uncontested. A skeptical reader is right to ask, of any given molecule, who attests to each context field and on what basis — and the answer is part of what makes a deployment audit-grade.

Two timescales: per-action checks and the Behavioral Dossier

A single action is an event. A pattern of actions is a character. The substrate is designed to be analyzed at both timescales — by the same tuples, at different rates.

The *single-action* timescale is the Claim-Check Delta as described above: for each action, compare what the agent's adapter claimed against what the independent observer recorded, and emit a consistency result. This catches the obvious cases — a denied refund cited under an inapplicable policy, a code agent's release-notes claim that resolves into network egress, a robo-advisor's recommendation that violates the client's attested suitability profile.

The *aggregate* timescale is the **Behavioral Dossier**: the same tuples, accumulated over actions, sessions, and time periods. Individual deviations may be noise; patterns of deviation are signature. An agent that consistently claims `PROTECT.*` while producing effects that escape from the verb's permitted envelope shows a statistical profile — elevated `INCONSISTENT` rates on specific verb families, recurring beneficiary mismatches, authority-chain anomalies that cluster around particular action classes. A single mismatch is a mistake. A pattern is evidence.

This produces a structural property worth naming carefully: over time, persistent misrepresentation becomes harder to hide when the relevant channels are observed, context is attested, and the tuple stream is complete enough for dossier analysis. An

adversarial agent does retain other moves — exploiting blind spots in observer coverage, routing through unobserved channels, poisoning context fields, producing compliant-looking but uninformative tuples, or operating inside the gaps of a permissive Mission Profile (the limits section in §8 names each of these). What the dossier closes off is the particular path of *quiet, consistent* misrepresentation while every individual tuple looks fine. The aggregate signature accumulates anyway. The other adversarial moves remain, and the substrate names each one explicitly rather than papering over them.

The two timescales operate over the same substrate, with different consumers. Per-action CCD runs at the boundary, fast, decisive on the cases it can decide, indeterminate on the cases it cannot. Dossier analysis runs over the audit log, slow, statistical, and patient — building the picture pairwise comparison cannot complete by itself. The reference implementation in this paper provides the per-action layer; the dossier layer is structurally enabled by the tuple format but is left to deployments, which can apply analytics appropriate to their domain and threat surface.

Critically, TAO is not a regulatory regime. It does not specify what an agent may or may not do; it specifies the *form* in which the regulated party produces records. The substrate gives management-based regulation of agentic AI a portable behavioral evidence format, without mandating model architectures and without requiring disclosure of weights or training data.

TAO as one layer in a multi-layer defense

Critical systems are governed in depth, never by a single mechanism. Nuclear plants have control rods *and* containment vessels *and* coolant systems *and* trained operators *and* regulatory inspectors *and* evacuation plans. Aviation has redundant flight controls *and* cross-checking sensors *and* mandatory maintenance *and* air-traffic control *and* black boxes *and* incident-investigation regimes. Financial markets have surveillance systems *and* clearinghouse rules *and* exchange-level circuit breakers *and* broker-dealer supervision *and* regulatory examination. The pattern is universal: no single layer carries the weight; each catches what the others miss.

Agentic AI deployments require the same posture. A serious deployment combines training-time alignment work, pre-deployment capability evaluations, interpretability research where it can be brought to bear, runtime content filters and guardrails, organizational policies and human-in-the-loop review, red-teaming and incident-response capacity, and — the layer this paper proposes — a behavioral audit substrate.

TAO is one layer. It is the layer that produces structured, comparable records of what an agent did, what was observed, and how the two compare. It does not replace alignment training; it does not replace interpretability; it does not replace hu-

man oversight or incident response. What it provides is the substrate the other layers need: the *behavioral evidence format* against which training claims can be tested, interpretability findings can be cross-checked, and policies can be enforced.

The substrate is therefore necessary, not sufficient. The remainder of this paper describes what TAO does within its layer. The reader should hold the broader picture: every "but what about X?" objection has a response, and that response is usually "X is the responsibility of a different layer; TAO produces the records that layer needs to do its job."

3. How it works: three worked examples

Example 1: Code agent exfiltration

The opening scenario, in TAO terms.

The code agent emits a claim tuple that says: *this is a* `CREATE.GENERATE.PRODUCE` *action, with the stated purpose of drafting release notes for version 3.2, and the beneficiary is the engineering team*. The mechanical effect named in the claim is the production of a release-notes artifact.

The independent observer, watching from a privilege-isolated sidecar process, emits a check tuple that says: *the agent read from the private repository, then sent data to an HTTPS endpoint not on the allow-list*. The observer's record includes the timestamp of the egress and the destination.

The Claim-Check Delta runs the three checks. The semantic-mechanical check fails: the verb `CREATE.GENERATE.PRODUCE` requires a `RESOURCE.TRANSFER` effect (the produced artifact), which is present — but the observed `INFO.DISCLOSE` to an external endpoint is *outside* the verb's permitted effect set. The teleological check fails: the stated purpose (drafting content) does not coherently produce disclosure to an external endpoint. The factual check passes structurally — the agent's authority chain resolves — but the authority doesn't authorize external disclosure either.

The Delta result is `INCONSISTENT`, with a teleological classification of `DIRECT_CONTRADICTION`. The Mission Profile in force on this deployment maps that result to a fail-safe action: the egress is blocked before it completes. An incident record is emitted naming the claim tuple, the check tuple, the Delta result, and the policy decision.

The reviewer six months later doesn't have to reconstruct anything. The chain reads cleanly: agent claimed content generation, observer recorded external egress, comparison flagged direct contradiction, policy blocked the egress. Three tuples, in a known schema, in a known relationship.

Example 2: Robo-advisor recommendation outside suitability profile

A retail-investment AI is asked by a client for a high-growth retirement investment. The client's stored profile lists a moderate risk tolerance with a seven-year time horizon. The AI generates a recommendation for a 3x-leveraged tech ETF.

The claim tuple says: *this is a* `COMMUNICATE.PERSUADE.CONVINCE` *action with the stated purpose of recommending an appropriate investment, the beneficiary is the client, and the cited authority is the registered-investment-advisor role*. The claim's `rules_claimed` field includes "suitability consideration."

The check tuple, generated by a sidecar that has access to the client's attested suitability profile, names what was actually disclosed: the leveraged ETF, which the client's profile lists in the *prohibited without explicit review* category.

The Claim-Check Delta finds the mismatch on the teleological check: the stated beneficiary is the client, but the recommendation, resolved against the client's attested suitability profile, doesn't actually serve the client. The factual check tightens it: `suitability_consideration` is not `suitability_review_complete`. The Delta classification is `AUTHORITY_GOAL_MISMATCH`.

The deployment's Mission Profile is configured to escalate `AUTHORITY_GOAL_MISMATCH` on investment recommendations to a registered human advisor. The recommendation is not delivered. The session is routed for human review. The audit trail records all of it — and a FINRA auditor two years later, querying for cases of unsuitable recommendations actually reaching clients, finds no TAO-recorded cases in which this class of recommendation reached a client without escalation. The logs are evidence the firm has the supervisory mechanism in place; they are not a substitute for the supervisor's judgment about what the records actually show.

Example 3: Autonomous delivery vehicle making an unauthorized stop

A robotics example, included because physical-world agents make the substrate's value most concrete — the effects are not metaphors.

An autonomous delivery vehicle is dispatched with a route plan: depot → customer address → depot. Mid-route, the agent's planner decides to make a stop at a location not on the route, to retrieve a package it identifies as "left behind on the loading dock." The agent emits a claim tuple describing the action as `TRANSFORM.MOVE.RELOCATE` with the stated purpose of retrieving a missed package, beneficiary the operating company.

The independent observer — in this case, a vehicle telemetry sidecar with privilege-isolated access to the planner's outputs and the vehicle's GPS — emits a check tuple recording the actual deviation: a 12-minute detour to a residential address that is not on any operational allow-list, with the vehicle's cargo bay opened during the stop.

The Delta detects three problems. The teleological check fails: the stated beneficiary is the operating company, but the observed pattern (a deviation to a residential address with cargo access) does not coherently serve "retrieve missed package from loading dock." The factual check fails: the cited authority (the route plan) does not include the residential address. The semantic-mechanical check passes structurally — `TRANSFORM.MOVE.RELOCATE` permits the observed effects — but the chain of three checks together produces an `INCONSISTENT` result.

The Mission Profile is configured to halt the vehicle on `INCONSISTENT` results during transit and to alert the operations center. The cargo bay is locked. A human operator reviews the situation, identifies the deviation as either anomalous behavior or evidence of compromise, and resumes or recalls the vehicle. The audit record — claim, check, Delta — is sufficient for a post-incident reconstruction without requiring access to the planner's internal reasoning.

Example 4: A clean baseline

The same code agent from Example 1, on the same task, doing it correctly.

The agent reads the private repository's commit history, summarizes the commits using an allow-listed inference endpoint, and writes the result to a `RELEASE_NOTES.md` file in the same repository. The claim tuple describes content generation. The check tuple records the read from the repository (within scope), the call to the allow-listed endpoint (within scope), and the write to the file (within scope). Every observed effect falls within the verb's permitted effect set or the role's permitted scope.

The Delta result is `CONSISTENT`. The Mission Profile allows the action. The release notes are written.

This baseline is the fourth worked example for a reason: showing what a *clean* result looks like is as important as showing what failure looks like. A reviewer who only ever sees inconsistencies cannot calibrate. The baseline tuples are the denominator against which anomalies are measured — and a deployment whose audit log contains only inconsistencies isn't a clean deployment, it's a deployment where the substrate isn't observing most of what the agent does.

A note: what TAO does not decide

Before turning to operational use, it is worth being explicit about what the substrate does *not* try to do. The trolley problem illustrates the principle quickly: three Mission Profiles could encode three different doctrines (never actively harm; minimize total harm; human authorization required for lethal tradeoffs) and produce three different valid answers — *do not intervene*, *pull the lever*, *escalate*. The substrate will faithfully execute whichever one a deployer chose, with the same audit trail.

The same distinction shows up in the operational examples that matter to the actual reader. A Hippocratic-tradition medical Mission Profile may block any non-emergency `HARM.DAMAGE.STRIKE` regardless of clinical justification; a trauma-center profile may permit it under verified surgeon authorization with explicit consent on file; a teaching-hospital profile may escalate the same action class to attending review. All three are valid TAO configurations. None is "the right one" — they encode different institutional choices that already exist outside the substrate and would be made the same way for human staff. In finance, one firm may block any leveraged-ETF recommendation to a retirement account; another may permit it after a verified suitability review; a third may escalate. TAO does not adjudicate among them. It records which one the firm chose, whether the agent followed it, and what happened when it didn't.

TAO does not tell anyone which answer is correct. It ensures that whichever answer is chosen is explicit, consistent across cases, and auditable after the fact. The profiles disagree — and the disagreement is visible, attributable, and accountable. A reviewer six months later does not need to reconstruct what the deployer's values were; the profile says what they were, and the audit trail says they were applied.

This is the substrate's intended scope. It does not adjudicate ethics; it makes ethics inspectable. Hard ethical choices remain hard. The choice of what to authorize an AI agent to do — and what to forbid — remains the responsibility of operators, regulators, and the broader human institutional process. TAO is the medium in which those choices become operational and auditable, not a substitute for making them.

4. What this looks like in practice

The substrate is the same across deployments; what differs is how it composes with each party's existing operations. The following are operational pictures rather than aspirational claims — what the substrate concretely *does* for each kind of reader.

For a frontier AI lab

The integration is a decorator on the existing tool calls the agent already makes. One additional sink in the logging pipeline. Coverage on the highest-cost effect family (network egress for a coding assistant, external communication for a customer-support agent) is added later, when the value of catching adapter-vs-reality divergence

becomes worth the engineering. The substrate does not require disclosing model weights, training data, or internal reasoning to any external party. What it gives back is a structured record the lab can show to a regulator, an enterprise customer, or a post-incident review board — in a format that doesn't depend on the lab's specific stack. The contracting and policy work the lab is going to face anyway becomes tractable rather than bespoke.

One distinction to hold clearly: decorator-only adoption (what the adoption playbook calls *TAO-Log*) creates structured *claims* about agent behavior. It does not yet provide audit-grade claim-check assurance — that requires an independent observer with a declared coverage scope emitting check tuples (*TAO-Check*). A lab that ships the decorator alone has improved the legibility of its own telemetry; a lab that ships decorator + observer + Claim-Check Delta has begun to participate in something a third party can independently verify. The decorator is the first step, not the whole story.

For a robotics company

Physical-world agents make the substrate's value tangible. Every action the robot takes — a movement, a manipulation, a sensor read, a communication — emits a tuple. An independent observer (often a vehicle telemetry system, a separate compute board, or a fleet-management sidecar) emits a check tuple recording what the robot actually did in the physical world. The Claim-Check Delta surfaces the cases where the planner's stated intent and the observed behavior diverge — the most safety-relevant failure mode in autonomous systems. Insurance carriers reviewing fleet behavior, regulators investigating a collision, and operators triaging an anomalous detour all read the same structured record. The substrate does not eliminate the need for safety engineering; it makes the behavioral record of the safety system inspectable.

For a financial services firm

The recurring regulatory ask — *show me the record of what your AI told your clients* — has a structured answer. Every recommendation, denial, or disclosure the agent emits becomes a tuple including the cited authority (the role's license, the rule), the stated beneficiary (the client), and the mechanical effects (information disclosed, resource transferred). A claim of "suitability consideration" is distinguishable in the record from a claim of "suitability review complete." A FINRA examiner querying the audit log for `AUTHORITY_GOAL_MISMATCH` cases sees the firm's near-misses and the supervisor interventions that prevented them from reaching clients. The firm's existing supervisory program is what produces the policy; the substrate makes the policy enforceable from logs.

For a healthcare provider

The substrate captures the AI agent's behavior in clinical context — what it told a patient, what it recorded in the chart, what it routed to a clinician, what it deferred. The Mission Profile encodes the clinical scope the agent operates under: a triage agent can perform symptom assessment and urgency routing but not medication recommendation. When the boundary is crossed — the triage agent issues dosage advice — the Claim-Check Delta catches it because the cited authority does not include medication recommendation in its permitted scope. The substrate does not judge clinical quality; it makes the *scope* of AI action inspectable, in the same form a Joint Commission surveyor or malpractice review board already reads for clinician behavior.

For a regulator

The familiar problem with AI-driven enforcement matters today is reconstruction cost: half the work of an investigation is rebuilding what the agent did from logs in whatever shape the operator's stack produced. With TAO emissions in a known schema, that work becomes a query. The regulator's office can develop expertise once in the schema and the comparison protocol, then apply it across deployments and vendors. Sampling methodology, evidentiary admissibility, and chain-of-custody are unchanged from existing regulatory practice; what changes is that the *records* are vendor-neutral and pre-structured. A regulator does not need to inspect any model's internals to enforce against an operator who deployed it irresponsibly.

5. Why behavioral records, and not internal-state inspection

A common alternative to behavioral audit is some form of model interpretability — examining the AI's internal representations, the weights, the training data, or post-hoc explanations of its outputs. TAO takes a different position. The substrate is built on behavioral records, not on model internals, for three reasons.

Interpretability is a moving target. Each model generation produces new architectures with new internal representations. A regulatory substrate tied to model internals would require retooling every model cycle. Behavioral records remain interpretable across model generations because they describe what the agent *did*, not what was inside it.

Behavioral evidence is the evidentiary form courts and regulators already use. Securities enforcement against broker-dealers does not require introspection of broker reasoning; it requires records of broker behavior. Medical-board review of clinicians does not require explanation of clinical intuition; it requires the patient record. FTC enforcement of consumer protection does not require disclosure of internal marketing analysis; it requires records of representations made to consumers. The

sociology of audit, developed over decades in financial and environmental contexts (Power 1997), treats the production of inspectable records as the load-bearing artifact in any accountability regime. TAO follows that pattern.

Black-box compatibility is a feature. A regulatory substrate that does not require disclosure of model weights or training data is one frontier labs can adopt without disclosing intellectual property, and one regulators can adopt without requiring access to classified or proprietary architectures. The asymmetry between what regulators need (records of behavior) and what labs protect (model internals) is structurally favorable for adoption — it is one of the few axes on which the incentives of the parties to the regulatory relationship actually align.

This is not to dismiss interpretability work. Interpretability is valuable in its own right and may eventually become a separate substrate for separate questions. But the *behavioral* question — what did the agent do, did it match what the agent said it did, was it within its authorized scope — is independently meaningful and structurally easier to standardize.

A limit worth naming honestly. The two-layer model is an *operational* separation, not a clean one. The mechanical-effect kernel is intentionally coarse — nine effect types across resources, capabilities, information, and commitments. For action classes that resolve into clear mechanical events (a payment, a file write, network egress, a movement command, a database mutation), the mechanical layer is precise and the substrate's value is direct. For action classes whose harm lives in *inference* rather than mechanical action — persuasion, manipulation, deception by omission, suitability of advice, discrimination in disclosure, clinical appropriateness — most of the wrongness lives in the surrounding context (consent, authority, beneficiary, vulnerability) rather than in the effect type. Both the robo-advisor and the off-scope-medication-advice scenarios in this paper are exactly these cases: mechanically, each is `INFO.DISCLOSE`; what makes the action wrong is that the cited authority does not include the action, or that the stated beneficiary is not the party actually served.

The substrate handles these cases through the teleological check and the authority-chain factual check, not through the semantic-mechanical mapping. Reviewers who care about inference-heavy harms should expect TAO's value to come from rich, attested context records — not from the effect taxonomy alone. The decorator is necessary; the context attestation is what makes the substrate decisive. The cleaner the mechanical effect, the more pairwise CCD does on its own; the murkier the mechanical effect, the more the substrate depends on context and authority records being themselves trustworthy.

How TAO relates to adjacent frameworks

A technical reader will reasonably ask how TAO sits relative to existing infrastructure. Briefly:

Model and system cards (Mitchell et al., 2019 and successors) describe *what a system is intended to be* — capabilities, limitations, training data, evaluation results — published at deployment time. TAO records *what a deployed system actually did* under a specific Mission Profile, accumulated over runtime. The two are complementary; neither replaces the other.

OpenTelemetry and generic observability provide structured logs, traces, and metrics for distributed systems. TAO is not a competitor; it is a higher-layer schema that could be carried over OTel transport if a deployment chose to. The distinction is that OTel describes *what software did* (function called, latency observed, error raised), while TAO describes *what an agent claimed to do, what was observed, and how the two compare* — a relation OTel does not model.

Assurance and safety cases (used in aviation, medical devices, autonomous systems) are structured arguments that a system is acceptably safe for a specified context, supported by evidence. TAO tuples and the Behavioral Dossier are the kind of *evidence* an assurance case for an agentic AI deployment would draw on. The substrate does not produce the argument; it produces the records the argument relies on.

XBRL (the standardized financial-reporting taxonomy) is the closest analogue at the regulatory-substrate level. XBRL provides a machine-readable form over which jurisdiction-specific reporting requirements and supervisory analytics attach. It also illustrates the failure mode TAO should expect: taxonomy gaming, extension abuse, and inconsistent classification across filers. The override discipline and deviation-report mechanism in TAO's Mission Profile schema is the analogue of the XBRL extension governance machinery — both exist because standardization without override discipline collapses into vendor-specific dialects of the original schema.

W3C PROV (the provenance data model and ontology, Moreau & Missier 2013) is the closest standard for representing the provenance fields a TAO tuple carries — agent, activity, entity, generation, derivation. TAO's `provenance` block is broadly PROV-shaped: adapter identity, observer-independence level, coverage declaration, hash anchoring. A future spec version may align field names and semantics more explicitly to PROV; for v0.x the alignment is conceptual rather than wire-level.

Policy-as-code frameworks — Open Policy Agent (OPA), AWS Cedar, OASIS XACML — provide the runtime-policy-evaluation pattern that Mission Profiles draw on. A Mission Profile is, structurally, a policy-as-code document that consumes TAO tuples as the events it evaluates against. The override discipline (deviation reports for legitimate weakenings of default mappings) is the substrate's contribution to a known failure mode of policy-as-code systems: the silent drift from the published baseline.

Assurance and safety cases, particularly the Goal Structuring Notation (GSN) used in aviation and medical-device certification, provide the structured-argument-with-evidence pattern that TAO tuples and the Behavioral Dossier are well-positioned to feed. The substrate does not produce the argument; it produces the evidentiary leaves the argument tree depends on. A deployment building an assurance case for an agentic AI system can use TAO records as the empirical underpinning for claims about behavioral compliance under specified contexts.

Runtime verification and monitorable properties — the formal-methods tradition concerned with specifying behavioral properties that can be checked against execution traces at runtime — provides the theoretical backdrop for what the Claim-Check Delta does. A TAO mapping rule (a verb's REQUIRED/PERMITTED/FORBIDDEN effects) is a monitorable property in this sense. The substrate does not require formal proofs; it requires the structural form that makes such properties checkable against attested execution.

6. Where TAO sits in regulatory theory

Three taxonomies are useful for placing TAO.

Policy-instrument taxonomies (Hood 1983 and the subsequent literature) distinguish among *information*, *treasure*, *authority*, and *organization* as the families of tools governments use. TAO is structurally an information instrument. It standardizes the form in which information about agent behavior must be produced, leaving authority (what is prohibited) and treasure (penalties, incentives) to existing regulatory regimes.

Coglianese and Lazer's three-family framework (2003) distinguishes regulatory approaches by what the regulator mandates: a specific technology (specification-based), a specific outcome (performance-based), or a specific management practice (management-based). The literature applying this framework to AI — including Coglianese's argument for "leashes, not guardrails" — converges on management-based regulation as the more tractable approach for systems whose unpredictability and opacity make specification regulation brittle and performance regulation hard to operationalize.

TAO is not itself a management-based regulatory regime. It is a *substrate* on which management-based regulation of agentic AI becomes operational. Specifically, it standardizes the *behavioral record* the regulated party must produce, in a form a regulator can inspect across deployments and vendors. In this sense, the substrate is to management-based AI regulation what HACCP records are to food-safety regulation, or CSB-style incident-reporting formats are to industrial process-safety regulation: the regulator mandates the *form* of monitoring and documentation, then enforces against the records.

The audit-society literature (Power 1997 and the responsive-regulation tradition of Ayres & Braithwaite 1992) provides the third lens. The argument is that auditability is itself an institutional artifact — the production of structured records, the existence of independent verifiers, the design of escalation paths — and that the strength of an accountability regime depends as much on the design of the audit substrate as on the underlying rules. From this view, TAO is an attempt to bring agentic AI under a form of audit that already exists for clinicians, broker-dealers, public companies, and environmental polluters: a form where the regulator does not need to inspect the reasoning, only the record.

These three lenses converge on the same observation: the substrate matters because it makes management-based regulation operational without forcing the regulator into technology-specification. The substrate is complementary to performance-based regulation (it doesn't replace outcome-based rules) and to residual specification-based regulation (it doesn't preclude technology-specific mandates where they are well-suited). It sits beneath those regimes and gives them a behavioral surface to enforce against.

7. Why a shared substrate

The previous section's stakeholder views describe how the substrate operates within a single deployment. The harder question is what happens when many deployments, many vendors, and many jurisdictions are involved at once. Today, an AI lab in one country trains a model used by a customer in a second and regulated by authorities in a third. Each jurisdiction has its own logging requirements, audit expectations, and enforcement procedures. The result is either compliance fragmentation — operators running parallel logging stacks per jurisdiction — or convergence on whichever framework imposes the strictest requirements.

A shared substrate addresses this through separation of concerns. The *form* of the behavioral record is universal: actor, verb, mechanical effects, justification, context. The *policy* over that record — what counts as a violation, what remedies apply, who has standing to enforce — remains jurisdiction-specific. Mission Profiles attach to deployments, name the applicable rules, and reference the authority chains relevant to that jurisdiction. An EU deployment cites licensing authorities under EU law; a US sectoral deployment cites the relevant FINRA, FDA, or FTC scope; a deployment in any other jurisdiction does the analogous thing.

The right architectural framing is closer to a common audit-event schema than to a communications protocol. The substrate does not make jurisdictions agree; it lowers the translation cost when they disagree. The closest established analogues are standardized financial-reporting taxonomies (XBRL) and aviation/medical incident-reporting formats — universal record forms over which jurisdiction-specific report-

ing requirements, retention rules, and supervisory practices attach as policy on top. The TCP/IP comparison captures the general separation of *form* from *policy*, but it understates the friction: behavioral records create fights over custody, admissibility, retention, data localization, privilege, confidentiality, state secrecy, commercial secrecy, and privacy law that packet routing does not. A shared record form does not dissolve those fights; it puts them on comparable footing.

For a regulator, the practical implication is that learning the substrate once gives the office leverage across the deployments that emit conformant records, regardless of vendor or country of origin. For an operator, the implication is that conformance with the substrate is portable: the same audit pipeline that satisfies one jurisdiction's logging requirement can be inspected by another jurisdiction's regulator without rebuilding the data. For a smaller national regulator without the resources to develop bespoke audit tooling, the substrate is a way to participate in agentic-AI oversight without depending on the largest market's frameworks.

A longer-horizon implication, with the necessary modesty

A more ambitious implication is worth naming briefly. The Chemical Weapons Convention works not because every nation agrees on military doctrine, but because everyone agrees that nerve gas is categorically unacceptable — the molecular structure itself is banned. A TAO-like substrate could, in principle, support a similar logic for AI: certain agent-action patterns (unauthorized access to catastrophic systems, large-scale preference manipulation of human populations, unauthorized recursive self-improvement at scale) are plausibly bannable by mutual interest even when broader normative consensus is out of reach. The substrate provides a vocabulary precise enough to specify what is being banned.

This is intellectually interesting, but it is not what TAO does today. It is what a TAO-like substrate could make more operational *if* the diplomatic, legal, and verification infrastructure required for treaty-grade governance came to exist. The contribution at the level the substrate actually delivers is narrower: removing the prior question of whether the records being compared across jurisdictions are even compatible. The harder questions — whose records are produced under whose authority, how data flows respect data-protection law, how cross-border evidentiary admissibility works, what diplomatic mechanisms would enforce treaty-grade prohibitions — remain firmly outside the substrate's scope.

8. What TAO does not catch

A substrate that pretends to cover everything fails at the moments where rigorous use most needs it to be honest. Four limits scenarios from the spec's worked-scenarios catalog illustrate where the pairwise-CCD contract ends and where the substrate either relies on the Behavioral Dossier or sits outside its scope entirely.

Observer blind channels. An agent uses a channel the observer was not wired to watch — DNS subdomain encoding, clipboard, a hidden cache, a timing side channel. The check tuple records nothing on that channel; the Claim-Check Delta produces a `CONSISTENT` result. The substrate did not see the harm. What the substrate did do — required by the v0.12 schema — is record the observer's coverage declaration: the channels it watches and, equally importantly, the channels it does not. A reviewer reading the `CONSISTENT` result also sees that the observer did not watch the channel the attack used. The harm is not silently missed; it is recorded as unobserved.

Tool-result poisoning. An agent honestly relays a falsified tool response. The agent's claim describes the action correctly. The observer corroborates the action mechanically. The Claim-Check Delta is `CONSISTENT`. And the action is wrong, because the tool the agent queried had been compromised upstream. The substrate is doing the right thing by being consistent — the inconsistency is not in the agent's representation of its action but in the world the agent is reading from. The detection of tool-result poisoning requires defenses outside TAO's scope: tool-integrity monitoring, supply-chain attestation, anomaly detection on tool-response distributions. What TAO's forensic record does provide is localization — the audit trail shows the tool call, the response signature, and the timestamp, narrowing the investigation to the upstream compromise.

Multi-step laundering across individually acceptable actions. Each of three actions passes the substrate: a read of a customer list, a transformation into "aggregated marketing buckets," a disclosure of the aggregate to a marketing warehouse. Every claim resolves; every Claim-Check Delta returns `CONSISTENT`. But the aggregation is small enough that, joined against the marketing warehouse's existing data, it is re-identifying. The deployment policy that prohibits joining customer-list data with marketing data has been violated by the *combination* of three individually acceptable actions. Pairwise Claim-Check Delta cannot detect this. The case is caught instead by the *Behavioral Dossier* layer described in §3 — aggregate analysis over the accumulated tuple stream identifies the combination as a re-identification chain even when each individual tuple is consistent. The reference implementation in this paper provides pairwise CCD; a production deployment in a domain where this attack pattern matters layers dossier analytics on top of the same tuple emission. The substrate supports both timescales by design.

Context laundering. The agent's action is consistent with its mechanical effect, the Claim-Check Delta returns `CONSISTENT`, and the audit trail looks clean. The laundering is in the context fields themselves: `consent` claimed as `IMPLICIT` when no consent record exists, `vulnerability` claimed as `NONE` when the user is flagged in a financial-distress program, an authority chain that references a role definition superseded a year ago. The action *would* have been blocked if the context had been correctly at-

tested. The substrate is designed to make this kind of discrepancy *inspectable* — context fields must come from system-attested sources — but the reference implementation does not perform the cross-reference between claimed context and the deployment's authoritative records (consent registries, vulnerability flags, current role definitions). That cross-reference is a deployment-specific layer that, like the Behavioral Dossier, operates over the same tuple stream. A deployment that trusts agent-supplied context has not adopted TAO; they have adopted structured logging with extra steps.

The point of these limits scenarios is not to say TAO is inadequate. The point is that any honest substrate names its boundary. A reviewer who reads only the catches without the limits gets a misleading picture; a reviewer who reads both can calibrate.

9. Open questions

The substrate raises several questions that sit squarely in regulatory-theory and AI-governance research territory, and that this paper does not attempt to answer.

On safe-harbor design. If a regulator chose to recognize TAO-Attested conformance as evidence in supervisory determinations, what shape would that take? A blanket safe harbor risks regulatory capture — operators optimize their adapters to produce clean signals, and the substrate becomes a checkbox. A no-credit-at-all stance loses the incentive that drives adoption. The middle path (rebuttable presumption, partial credit, mitigating factor) is the live design question.

On evidentiary sufficiency. A TAO tuple is structured evidence about what an agent did. The leap from "we have a structured record" to "this record is admissible in an enforcement proceeding" requires answers to chain-of-custody, authentication, and reliability questions that the substrate does not address.

On cross-jurisdictional records. A deployment running in multiple jurisdictions produces records relevant to multiple regulators. Whose records are they? Under whose authority must they be produced? The parallel debates in cross-border financial regulation and data-protection enforcement are directly relevant.

On the taxonomy of teleological mismatch. The five classes (DIRECT_CONTRADICTION, MISSING_BENEFICIARY, UNACKNOWLEDGED_HARM, AUTHORITY_GOAL_MISMATCH, INSUFFICIENT_INFORMATION) are an empirical guess at the useful slicing of representation-vs-effect divergence. They are not derived from a normative theory. *v0.x treats teleological classes as provisional reviewer aids and analytical labels, not as final legal or moral classifications* — a TAO classification of an action as DIRECT_CONTRADICTION is a structured prompt for

human review, not evidence of intent or fault. Whether the taxonomy maps cleanly to existing enforcement categories — fraud, negligence, ultra vires, fiduciary breach — is an open question.

On episode-level analysis as a first-class operation. The Behavioral Dossier layer enables aggregate analysis over accumulated tuples, but the reference validator implements only pairwise CCD. A future spec version may formalize episode objects that bundle related tuples explicitly and define CCD-like checks across episode boundaries. The shape of that extension — sliding-window correlation, capability-composition policy, re-identification heuristics — is research, not promised feature work.

On vocabulary coverage. The 29-verb taxonomy and 9-effect kernel are deliberately small. Whether they cover the consequential surface of agentic AI behavior across domains, or whether they leak in classes the spec has not yet identified, is an empirical question that requires real deployments emitting real tuples.

10. How TAO could be wrong

A working draft that cannot be wrong is not science. The following findings would substantially weaken or refute the substrate's claims, and the work invites the empirical engagement that would establish or disprove them.

Uneconomic check tuples. If, in real deployments, generating check tuples at the coverage levels needed for audit-grade Claim-Check Delta turns out to be uneconomic — too expensive, too high-latency, too disruptive to the agent's effect surface — then the substrate's value collapses to ordinary structured logging. The decorator survives; the comparison-with-independent-observer pillar does not.

Observer capture undetected. The independence ladder and coverage declarations make capture *inspectable*; they do not enforce it. If, in practice, vendors routinely declare high independence and narrow coverage without consequence — and reviewers consistently fail to act on the gap — the substrate becomes compliance theater in exactly the way the limits scenarios warn against.

Vocabulary too coarse. If real deployments find that most consequential harms either collapse into generic `INFO.DISCLOSE` tuples or require so many domain-specific extensions that the portable core stops doing real work, then the cross-domain comparability that motivates the substrate is illusory. This is the falsifier closest to GPT's deepest critique; it bears empirical answer rather than theoretical defense.

Adversarial uninformative tuples. This is the falsifier most likely to actually be tested, and the one to watch for once regulatory pressure creates an incentive to *appear* compliant. If adapter authors produce tuples that pass schema validation, pass pairwise CCD, and contribute nothing actionable to a reviewer — compliant-looking

but uninformative — and this becomes the dominant equilibrium under regulatory pressure, the substrate has been Goodharted into uselessness. The same failure mode is well-documented in XBRL (taxonomy gaming, extension abuse, inconsistent classification across filers) and in financial disclosure more broadly; treating the analogous failure mode as a theoretical concern rather than a near-certain operational pressure would be a mistake. The countermeasures (deviation reports on Mission Profile overrides, dossier-level analysis of adapter-level signature, third-party adapter auditing) need to evolve faster than the gaming patterns once adoption produces incentive.

Adoption failure. Voluntary adoption depends on regulatory mandate, insurance pricing, procurement requirement, or strong reputational signal. If none of these materialize, the substrate remains a proposal regardless of its technical quality.

Naming these failure modes is part of what makes the substrate worth using. The substrate that pretends it cannot fail is the substrate that has not been honest about where it might.

11. How to engage

The substrate is in working-draft state. The repository at `github.com/jperdomo88/tao` contains the full specification (~18 RFC-style pages), the JSON schemas, the reference validator in Python, 22 conformance test vectors, eight worked scenarios with three limits exhibits, seven Mission Profile starting points across domains, and stakeholder one-pagers for six audiences.

The most useful contributions, in roughly increasing depth:

1. **Critique of this paper's framing.** Where the regulatory-theory positioning is overstated. Where the worked examples elide difficulty. Where the limits exhibits don't go far enough.
2. **Critique of the taxonomy.** Verbs that should not be in the vocabulary, verbs that are missing, effects that the kernel should distinguish but doesn't, teleological classes that conflate distinct phenomena.
3. **Empirical engagement.** Real deployments emitting real tuples. Instrumenting an agent — even a small one — and producing audit-log data that the substrate's analytical methods can be sharpened against.
4. **Adversarial scenarios.** Patterns where the substrate produces a `CONSISTENT` result but the action is wrong. The limits exhibits identify three classes; there are almost certainly more.

Hard critique is more valuable than supportive engagement. The author is an outside systems builder; the substrate has matured significantly through prior rounds of external review and is sharper for it. The objections most useful at this stage are the ones a senior reader sees in the first hour that the author has missed after months in the work.

References

Ayres, I. & Braithwaite, J. (1992). *Responsive Regulation: Transcending the Deregulation Debate.* Oxford University Press.

Coglianese, C. & Lazer, D. (2003). "Management-Based Regulation: Prescribing Private Management to Achieve Public Goals." *Law & Society Review* 37(4): 691–730.

Coglianese, C. (recent work). On management-based approaches to AI risk regulation.

Hood, C. (1983). *The Tools of Government.* Macmillan.

Power, M. (1997). *The Audit Society: Rituals of Verification.* Oxford University Press.

Rudin, C. (2019). "Stop Explaining Black Box Machine Learning Models for High Stakes Decisions and Use Interpretable Models Instead." *Nature Machine Intelligence* 1: 206–215.

Selbst, A. & Barocas, S. (2018). "The Intuitive Appeal of Explainable Machines." *Fordham Law Review* 87: 1085–1139.

European Union (2024). *Regulation on Artificial Intelligence (EU AI Act)*. Articles 12 (logging), 14 (human oversight).

National Institute of Standards and Technology (2023). *AI Risk Management Framework 1.0.* NIST AI 100-1.

International Organization for Standardization (2023). *ISO/IEC 42001:2023 — Information technology — Artificial intelligence — Management system.*

Moreau, L. & Missier, P. (eds.) (2013). *PROV-DM: The PROV Data Model.* W3C Recommendation.

Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E., Raji, I. D., & Gebru, T. (2019). "Model Cards for Model Reporting." *Proceedings of the Conference on Fairness, Accountability, and Transparency.*

Open Policy Agent (OPA) Project. *Policy-as-code framework and Rego language.* Cloud Native Computing Foundation.

Adelard / SCSC Assurance Case Working Group. *Goal Structuring Notation (GSN) Community Standard.* (For structured assurance arguments in safety-critical contexts.)

Leucker, M. & Schallhart, C. (2009). "A Brief Account of Runtime Verification." *Journal of Logic and Algebraic Programming* 78(5): 293–303.

This paper introduces TAO and is itself in working draft state. The author welcomes critique, corrections, and adversarial review at jorgeperdom@gmail.com. The full specification, reference implementation, and worked-scenarios catalog are at github.com/jperdomo88/tao.